

Title

Multi-Echelon Supply Chain Network Design via Mixed-Integer Linear Programming (MILP)

Week-3 Business Analytics (BA) Task

Student Name: _Savitha M_____

Department: B.Sc. Computer Science – AI & ML

Task: Multi-Echelon Supply Chain Network Design

1.Introduction

Supply chain network design is an important optimization problem that helps organizations determine how products should be transported from factories to warehouses and finally to customer zones. This project uses Mixed-Integer Linear Programming (MILP) to design an efficient multi-echelon logistics network while minimizing total transportation and operating costs.

2.Problem Statement

The objective of this project is to design a global logistics network consisting of factories, warehouses, and customer zones. The model must consider factory production capacities, warehouse storage capacities, transportation limits, customer demand, and minimum service level requirements. The MILP model determines the optimal flow of products through the network while minimizing the total supply chain operating expenditure.

3.Network Design

The proposed supply chain network consists of two factories, two warehouses, and three customer zones. Products are transported from factories to warehouses and then from warehouses to customer zones. The network is designed to satisfy customer demand while respecting factory and warehouse capacity constraints and minimizing total transportation cost.

4.Input Data

The following data is used to represent the proposed multi-echelon supply chain network.

Factory Capacity

Factory	Capacity (units)
F1	100
F2	150

Warehouse Capacity

Warehouse	Capacity (units)
W1	120
W2	100

Customer Demand

Customer Zone	Demand (units)
C1	80
C2	70
C3	60

Minimum Service Level: 90%

Transportation Cost

Factory → Warehouse Cost (per unit)

	W1	W2
F1	4	6
F2	5	3

Warehouse → Customer Cost (per unit)

	C1	C2	C3
W1	4	6	5
W2	7	4	6

Note: The transportation costs are assumed values used for demonstrating the MILP optimization model.

5. Decision Variables

The following decision variables are defined for the MILP model:

x_{ij} = Quantity of products transported from Factory i to Warehouse j .

y_{jk} = Quantity of products transported from Warehouse j to Customer Zone k .

These variables determine the optimal flow of products through the multi-echelon supply chain network.

6.Objective Function

The objective is to minimize the total supply chain transportation cost from factories to warehouses and from warehouses to customer zones.

$$\text{Minimize } Z = \sum_i \sum_j C_{ij} x_{ij} + \sum_j \sum_k D_{jk} y_{jk}$$

Where:

- C_{ij} = Transportation cost from Factory i to Warehouse j
- D_{jk} = Transportation cost from Warehouse j to Customer k
- x_{ij} = Factory-to-Warehouse quantity
- y_{jk} = Warehouse-to-Customer quantity
- Z = Total transportation cost

7. Constraints

The MILP model includes constraints to ensure that factory capacities, warehouse capacities, product flow, customer demand, and minimum service levels are satisfied.

7.1 Factory Capacity Constraints

The total quantity shipped from each factory cannot exceed its available production capacity.

Constraints:

$$\sum_j x_{ij} \leq \text{Factory Capacity}_i$$

For example:

F1 capacity $\leq$ 100 units

F2 capacity $\leq$ 150 units

7.2 Warehouse Storage Capacity Constraints

The total quantity entering each warehouse cannot exceed its storage capacity.

Constraint:

$$\sum_i x_{ij} \leq \text{Warehouse Capacity}_j$$

For example:

W1 capacity $\leq$ 120 units

W2 capacity $\leq$ 100 units

7.3 Warehouse Flow Balance Constraints

The quantity entering a warehouse must equal the quantity leaving that warehouse to customer zones.

Constraint:

$$\sum_i x_{i\Box} = \sum_{\Box} y_{\Box\Box}$$

This ensures that there is no unexplained surplus or shortage at the warehouse level.

7.4 Customer Demand Constraints

Each customer zone must receive enough products to satisfy its demand.

Constraint:

$$\sum_{\Box} y_{\Box\Box} \geq \text{Customer Demand}_{\Box}$$

Therefore:

C1 demand = 80 units

C2 demand = 70 units

C3 demand = 60 units

7.5 Minimum Service Level Constraint

A minimum service level of 90% is required for each customer zone.

Constraint:

$$\sum_{\Box} y_{\Box\Box} \geq 0.90 \times \text{Customer Demand}_{\Box}$$

This ensures that at least 90% of the required customer demand is served.

8. Constraint Matrix

The constraint matrix represents the major restrictions of the supply chain network. Each row represents a constraint and each column represents the relevant decision variables.

Constraint	Mathematical Representation	Purpose
Factory Capacity	$\sum_j x_{ij} \leq F_i$	Limits shipments from factories
Warehouse Capacity	$\sum_i x_{ij} \leq W_j$	Limits warehouse storage
Flow Balance	$\sum_i x_{ij} = \sum_k y_{jk}$	Maintains warehouse flow
Customer Demand	$\sum_j y_{jk} \geq D_k$	Satisfies customer demand
Service Level	$\sum_j y_{jk} \geq 0.90D_k$	Ensures minimum 90% service
Non-negativity	$x_{ij}, y_{jk} \geq 0$	Prevents negative shipment quantities

Matrix interpretation

The constraint matrix ensures that all operational limitations are respected while determining the

optimal transportation plan. The model searches for the feasible solution with the minimum total transportation cost.

9. Python MILP Implementation

The MILP model is implemented in Python using the PuLP optimization library. The model defines factories, warehouses, customer zones, capacities, demands, transportation costs, decision variables, objective function, and operational constraints. The optimization solver determines the minimum-cost transportation plan.

10. MILP Mathematical Formulation

Decision Variables

Let x_{fw} represent the quantity transported from factory f to warehouse w , and let y_{wc} represent the quantity transported from warehouse w to customer zone c .

Objective Function

The objective is to minimize the total transportation cost:

Minimize:

$$\text{Total Cost} = \sum \text{Cost}(F,W) \times x(F,W) + \sum \text{Cost}(W,C) \times y(W,C)$$

Factory Capacity Constraints

The total quantity shipped from each factory must not exceed its available capacity.

$$\sum x(F,W) \leq \text{Factory Capacity}$$

Warehouse Capacity Constraints

The total quantity received by each warehouse must not exceed its storage capacity.

$$\Sigma x(F,W) \leq \text{Warehouse Capacity}$$

Warehouse Flow Balance Constraints

For each warehouse, the total quantity received must equal the total quantity shipped to customers.

$$\Sigma x(F,W) = \Sigma y(W,C)$$

Customer Demand Constraints

The quantity supplied to each customer must satisfy its demand.

$$\Sigma y(W,C) \geq \text{Customer Demand}$$

Minimum Service Level Constraints

A minimum service level of 90% is required for each customer zone.

$$\Sigma y(W,C) \geq 0.90 \times \text{Customer Demand}$$

The MILP model uses these constraints to determine the optimal transportation flows while minimizing the total supply chain transportation cost.

11. Optimization Results

The MILP model was successfully solved using the CBC optimization solver through the PuLP library. The solver returned an optimal solution with a minimum total transportation cost of 1680 cost units. All customer demands were fully satisfied, resulting in a 100% service level for all customer zones.

Factory to Warehouse Shipments

Route	Quantity
F1 → W1	100 units
F2 → W1	10 units
F2 → W2	100 units

Warehouse to Customer Shipments

Route	Quantity
W1 → C1	80 units
W1 → C3	30 units
W2 → C2	70 units
W2 → C3	30 units

Customer Service Level

Customer	Demand	Supplied	Service Level
C1	80	80	100%

C2	70	70	100%
C3	60	60	100%

12. Conclusion

This project successfully demonstrates the use of Mixed-Integer Linear Programming for multi-echelon supply chain network design. The model determines the optimal flow of products from factories through warehouses to customer zones while considering capacity, demand, flow balance, and minimum service-level constraints. The optimized solution achieves a minimum transportation cost of 1680 cost units and satisfies 100% of customer demand. Therefore, MILP provides an effective approach for minimizing supply chain operating expenditures while maintaining efficient customer service.

13. Python Source Code

```
from pulp import LpProblem, LpMinimize, LpVariable, lpSum, LpStatus, value

# Create the optimization model
model = LpProblem("Multi-Echelon_Supply_Chain", LpMinimize)

# Sets
factories = ["F1", "F2"]
warehouses = ["W1", "W2"]
customers = ["C1", "C2", "C3"]

# Factory capacities
factory_capacity = {
    "F1": 100,
    "F2": 150
}

# Warehouse capacities
```

```

warehouse_capacity = {
    "W1": 120,
    "W2": 100
}

# Customer demands
demand = {
    "C1": 80,
    "C2": 70,
    "C3": 60
}

# Factory to Warehouse transportation costs
cost_FW = {
    ("F1", "W1"): 4,
    ("F1", "W2"): 6,
    ("F2", "W1"): 5,
    ("F2", "W2"): 3
}

# Warehouse to Customer transportation costs
cost_WC = {
    ("W1", "C1"): 4,
    ("W1", "C2"): 6,
    ("W1", "C3"): 5,
    ("W2", "C1"): 7,
    ("W2", "C2"): 4,
    ("W2", "C3"): 6
}

# Decision variables
x = LpVariable.dicts(
    "Factory_to_Warehouse",
    [(f, w) for f in factories for w in warehouses],
    lowBound=0
)

y = LpVariable.dicts(

```

```

        "Warehouse_to_Customer",
        [(w, c) for w in warehouses for c in customers],
        lowBound=0
    )

# Objective function: Minimize total transportation cost
model += (
    lpSum(cost_FW[f, w] * x[f, w]
          for f in factories for w in warehouses)
    +
    lpSum(cost_WC[w, c] * y[w, c]
          for w in warehouses for c in customers)
)

# Factory capacity constraints
for f in factories:
    model += lpSum(x[f, w] for w in warehouses) <= factory_capacity[f]

# Warehouse capacity constraints
for w in warehouses:
    model += lpSum(x[f, w] for f in factories) <=
warehouse_capacity[w]

# Warehouse flow balance
for w in warehouses:
    model += (
        lpSum(x[f, w] for f in factories)
        ==
        lpSum(y[w, c] for c in customers)
    )

# Customer demand constraints
for c in customers:
    model += lpSum(y[w, c] for w in warehouses) >= demand[c]

# Minimum service level: 90%
for c in customers:
    model += lpSum(y[w, c] for w in warehouses) >= 0.90 * demand[c]

```

```

# Solve the model
model.solve()

# Display results
print("=====")
print("MULTI-ECHELON SUPPLY CHAIN OPTIMIZATION")
print("=====")

print("Status:", LpStatus[model.status])
print("Minimum Total Transportation Cost:", value(model.objective))

print("\nFactory to Warehouse Shipments:")
for f in factories:
    for w in warehouses:
        if x[f, w].value() > 0:
            print(f"{f} -> {w} = {x[f, w].value()} units")

print("\nWarehouse to Customer Shipments:")
for w in warehouses:
    for c in customers:
        if y[w, c].value() > 0:
            print(f"{w} -> {c} = {y[w, c].value()} units")

print("\nCustomer Service:")
for c in customers:
    supplied = sum(y[w, c].value() for w in warehouses)
    service = (supplied / demand[c]) * 100
    print(f"{c}: {supplied} units supplied / {demand[c]} units demand  
= {service:.1f}%")

```

13. References

1. Python Software Foundation. Python Documentation.
2. PuLP Documentation. PuLP: Linear Programming in Python.
3. COIN-OR. CBC (Coin-or branch and cut) Optimization Solver.
4. IBM. Mixed-Integer Linear Programming (MILP) Concepts.
5. Supply Chain Network Design and Transportation Optimization – Concepts and Methods.

